

SCMS — Exam Presentation Notes

Smart Complaint Management System

MCA II Semester Project · RVCE · Team: Pavan · Prabhava · Prem · Pramath

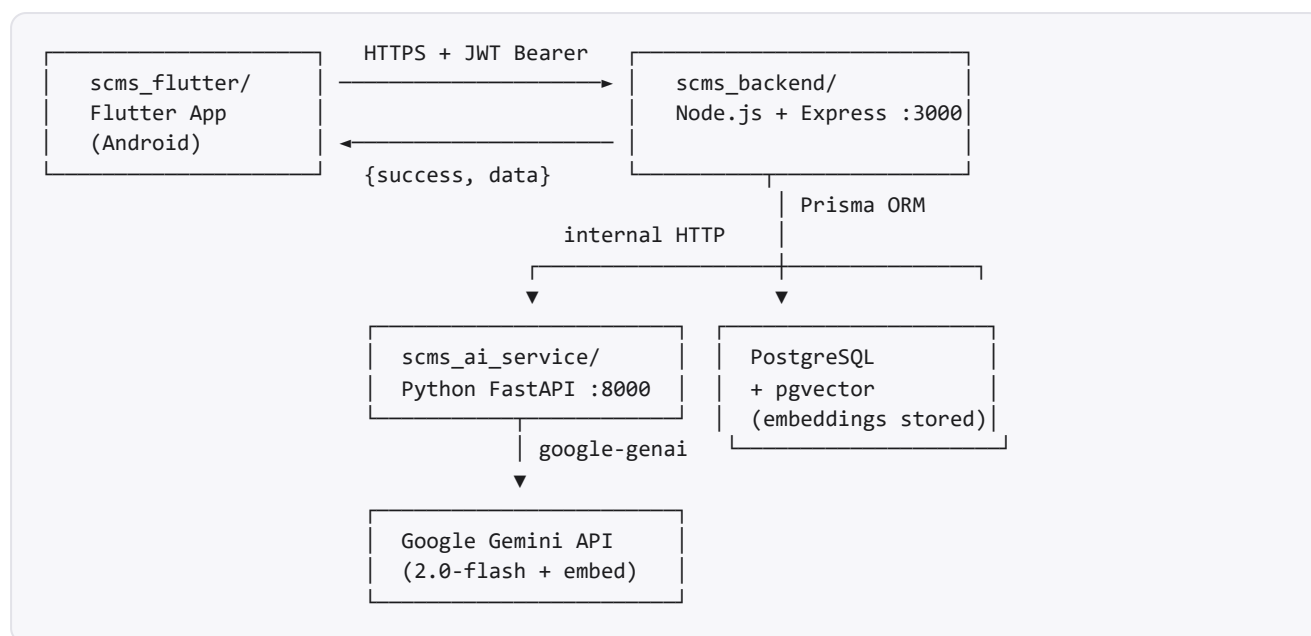
How to use this document: Each section is a module. Read the "One-line pitch" first, then the "How it works", then be ready to answer the "Likely exam questions". The last section tells the examiner exactly who built what.

1. The 30-Second Elevator Pitch (memorize this)

"SCMS is a **campus complaint management app** for RVCE. A student photographs a problem (broken light, water leak), the photo is **auto-stamped with GPS + timestamp**, and **Google's Gemini AI** fixes the grammar, auto-categorizes the complaint, and detects if a **duplicate** already exists. The complaint then flows through a **multi-role approval chain** — Student Representative → Admin → Staff — with **push notifications** at every step, an **SLA timer**, and **proof-of-resolution photos**. It's built as **three services**: a Flutter mobile app, a Node.js backend, and a Python AI microservice."

The 3 things that make it special (say these): 1. **AI-powered** — grammar correction, auto-categorization, and duplicate detection using Google Gemini + vector embeddings. 2. **Trustworthy evidence** — GPS + datetime watermark burned onto every photo; staff must upload proof photos to resolve. 3. **Full accountability chain** — 7-stage lifecycle with role-based approvals, SLA breach tracking, and ratings.

2. System Architecture (draw this diagram on the board)



Golden rules of the architecture (examiners love these): - **Flutter NEVER calls the Python AI service directly.** It always goes through Node.js (`/api/ai/*` → `aiProxy.js`). This keeps one security boundary and one auth layer. - **The AI service is "best-effort".** If Gemini or the AI service is down, the backend returns safe defaults and the app still works. AI **augments**, it is not a hard dependency. - **Auth is Google OAuth 2.0 only**, locked to the `rvce.edu.in` domain. There is **no** email/password login.

Why 3 services (microservice justification): - Separation of concerns — mobile UI, business logic, and AI/ML each scale and deploy independently. - Language fit — Python has the best AI/ML ecosystem; Node.js is excellent for I/O-heavy REST APIs; Flutter gives one codebase for mobile. - Fault isolation — AI service crashing doesn't take down complaint submission.

2.5 Tech Stack — What We Chose and WHY (viva gold — study this table)

Examiners almost always ask "why did you choose X and not Y?" for every layer. Here is a defensible answer for each choice, with the alternatives we rejected.

Layer	We chose	Why we chose it	Alternatives we rejected & why
Mobile app	Flutter (Dart)	One codebase → Android (and iOS-ready). Native performance (compiles to ARM). Rich widget set gave us the polished iOS-clean UI. Hot reload = fast development.	React Native — JS bridge is slower, less consistent UI across devices. Native Android (Kotlin) — would need a separate iOS codebase; double the work for a 4-person team.
State management	BLoC / Cubit	Clear separation of UI and logic, predictable & testable, industry standard for large Flutter apps. Scales well across many screens/roles.	setState — doesn't scale, logic leaks into widgets. Provider/Riverpod — good, but BLoC's event→state flow is more structured for a workflow-heavy app.
Backend API	Node.js + Express	Non-blocking I/O is ideal for a REST API that's mostly network/DB waiting. Huge npm ecosystem (JWT, Multer, Prisma, Firebase). Same language (JS) family as Flutter's JSON world — easy for the team. Fast to build.	Django/Flask (Python) — we reserved Python for the AI service; keeping the API in Node avoids mixing concerns. Spring Boot (Java) — heavier, slower to iterate for a student project.
AI microservice	Python + FastAPI	Python is <i>the</i> AI/ML language — Google's google-genai SDK, NumPy for vector math. FastAPI is async, fast, and auto-generates API docs (<code>/docs</code>).	Doing AI inside Node — poor ML tooling, no clean NumPy equivalent. Flask — synchronous by default, no built-in schema validation like FastAPI's Pydantic.
AI model	Google Gemini (<code>2.0-flash</code> + <code>embedding-004</code>)	Generous free tier (important for a student project), strong at grammar/classification, native embedding model for duplicate detection, simple SDK.	OpenAI GPT — paid, needs billing setup. Self-hosted LLM — too heavy to run on our hardware.
Database	PostgreSQL	Relational data (users, complaints, departments — lots of foreign keys). ACID guarantees for the workflow. Critically, the pgvector extension lets us store AI embeddings and do similarity search in the same DB — no separate vector database needed.	MongoDB — no strong relations/joins, no native vector search then; our data is highly relational. MySQL — no first-class vector extension like pgvector.
ORM	Prisma	Type-safe queries, auto-generated client, clean migration system, readable schema file. Speeds up development and prevents SQL mistakes.	Raw SQL — error-prone, no type safety. Sequelize — older API, less ergonomic than Prisma.
Auth	Google OAuth 2.0 + JWT	Campus already uses Google Workspace (<code>rvce.edu.in</code>) — students don't create new passwords, and we restrict signups to the college domain automatically. JWT = stateless, scalable sessions.	Email/password — password storage risk, no domain restriction, more friction. Session cookies — stateful, harder to scale across services.
Vector search	pgvector	Cosine-similarity search inside PostgreSQL; one database to manage.	Pinecone / Weaviate — external paid vector DBs, extra infrastructure and cost.

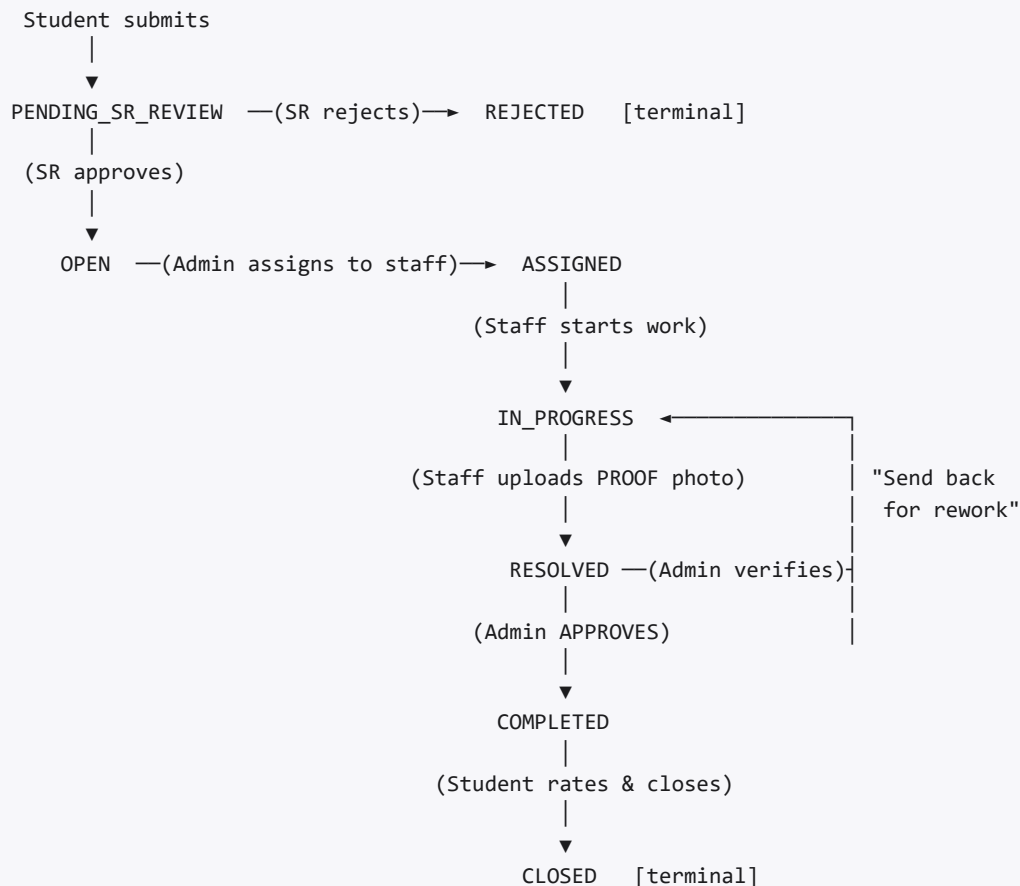
Layer	We chose	Why we chose it	Alternatives we rejected & why
Notifications	Firebase Cloud Messaging (FCM)	Free, reliable, the standard for Android push. Integrates with the Flutter Firebase SDK.	Custom WebSocket server — we'd have to build and host delivery/retry ourselves.
Offline storage	Hive	Lightweight, fast, pure-Dart key-value DB for saving complaint drafts offline.	SQLite — overkill for simple draft storage. SharedPreferences — not meant for structured objects.
Media upload	Multer + local storage	Simple multipart handling; local disk is fine for a campus-scale project.	AWS S3 — added cost/complexity beyond the project scope.

The 30-second stack summary (say this if asked "what's your tech stack?"):

"Flutter with BLoC for the mobile app; Node.js + Express with Prisma for the REST API; Python + FastAPI for the AI microservice calling Google Gemini; PostgreSQL with the pgvector extension for data **and** AI embeddings; Google OAuth for auth; and Firebase Cloud Messaging for push notifications."

3. The Complaint Lifecycle (THE most important thing to know)

This is the heart of the project. **Draw this state machine.**



7 statuses: `PENDING_SR_REVIEW` → `OPEN` → `ASSIGNED` → `IN_PROGRESS` → `RESOLVED` → `COMPLETED` → `CLOSED` (+ `REJECTED`).

Key rules to explain: - **SR (Student Representative)** acts as a first filter — approves valid complaints, rejects spam/invalid ones with a reason. - **Admin** assigns complaints to the right **Staff** member and later **verifies** the resolution. - **Staff cannot just mark "Resolved"** — they must upload a **proof-of-resolution photo**. No proof = HTTP 400 rejection. - **Admin verification loop** — Admin either **Approves** (→ `COMPLETED`) or **Sends back for rework** (→ `IN_PROGRESS`, same staff re-notified). - Only after **COMPLETED** can the student **rate** (1–5 stars) and close the complaint. - **SLA timer** runs in the background; a cron job marks complaints **SLA-breached** and notifies admins.

4. MODULE — Python AI Service (`scms_ai_service/`) 🧑‍💻 **Pramath**

One-line pitch: A FastAPI microservice with 4 AI endpoints powered by Google Gemini, designed to **fail safe** (never crash the app).

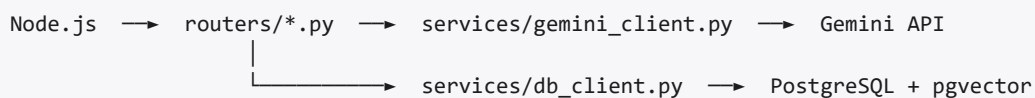
Tech stack: Python · FastAPI · Uvicorn · `google-genai` SDK · `psycpg2` + `pgvector` · `Pydantic` · `NumPy`

Models used: - `gemini-2.0-flash` → text tasks (grammar + categorization) - `models/gemini-embedding-004` → **768-dimensional** embeddings (for duplicate detection)

The 4 Endpoints

Endpoint	What it does	How
<code>POST /grammar-check</code>	Fixes spelling/grammar in the complaint text	Sends text to Gemini, returns corrected text + word-level diffs (EQUAL / DELETE / INSERT) so the app can highlight changes
<code>POST /categorize</code>	Auto-detects category (Electrical, Plumbing...) + severity	Gemini in JSON mode returns structured <code>{categoryName, severity, confidence}</code>
<code>POST /embed</code>	Converts complaint text → a 768-d vector, stores it in Postgres	Called by Node.js after the complaint row exists (needs <code>complaintId</code>). Stored in <code>complaints.embedding</code>
<code>POST /check-duplicate</code>	Finds similar existing complaints	Cosine similarity search via pgvector; returns matches above <code>SIMILARITY_THRESHOLD</code>

How it works (the flow)



- `main.py` — FastAPI app, mounts the 4 routers, exposes `GET /health`. On startup runs `ensure_embedding_column()` to guarantee the pgvector column exists.
- `models/schemas.py` — Pydantic request/response models for all endpoints.
- Fail-safe design:** every endpoint is wrapped in try/except. If Gemini fails, grammar returns "no corrections", categorize returns a default, duplicate returns "no duplicates". **The service never returns a 500.**

How duplicate detection works (explain the AI concept)

- Every complaint's description is converted into a **768-number vector** ("embedding") that captures its meaning.
- When a new complaint arrives, we compute **cosine similarity** between its vector and all existing vectors.
- Similarity close to **1.0** = nearly identical meaning → flagged as a duplicate.
- This catches duplicates even when wording differs ("light broken" vs "tube light not working").

Likely exam questions

- What is pgvector?* → A PostgreSQL extension that stores vectors and does fast similarity search.
- What is an embedding?* → A numeric representation of text meaning; similar texts have similar vectors.
- Why fail-safe?* → AI is an enhancement; the core complaint system must keep working if AI is down.

5. MODULE — Node.js Backend (`scms_backend/`) 🧑💻 *Prem*

One-line pitch: The central REST API that owns all business logic, authentication, the database, notifications, and background jobs. Everything flows through here.

Tech stack: Node.js · Express · Prisma ORM · PostgreSQL · JWT · Google Auth Library · Firebase Admin (FCM) · Multer (uploads) · ExcelJS · node-cron

The Response Envelope (important — caused real bugs)

Every response is wrapped by `sendSuccess` / `sendError` :

```
{ "success": true, "data": { ... } }  
{ "success": false, "error": { "message": "...", "code": 500 } }
```

The Flutter side has an interceptor that strips this envelope automatically.

Authentication flow (explain step-by-step)

1. Flutter → Google Sign-In (restricted to rvce.edu.in) → gets Google ID token
2. Flutter → POST `/api/auth/google` with that token
3. Backend verifies token via `google-auth-library`
4. Backend issues its OWN access JWT + refresh JWT (`jwtHelper.js`)
5. Flutter stores tokens; attaches "Bearer <token>" to every request
6. On 401, Flutter auto-calls POST `/api/auth/refresh`

Roles: `ROLE_USER` (student), `ROLE_STAFF`, `ROLE_SR`, `ROLE_ADMIN`. Guarded by `requireRole.js` middleware.

Backend structure

Folder	Responsibility
<code>routes/</code>	One router per resource: <code>auth</code> , <code>complaints</code> , <code>sr</code> , <code>analytics</code> , <code>ai</code> , <code>departments</code> , <code>categories</code> , <code>tags</code> , <code>zones</code> , <code>users</code>
<code>middleware/</code>	<code>authenticate.js</code> (JWT verify), <code>requireRole.js</code> (RBAC), <code>upload.js</code> (Multer media), <code>validateBody.js</code> , <code>errorHandler.js</code>
<code>services/</code>	<code>aiProxy.js</code> (calls Python AI), <code>googleAuth.js</code> , <code>fcm.js</code> (push notifications), <code>storage.js</code> (media files), <code>complaintNumber.js</code> (human IDs like SCMS-2026-00007)
<code>jobs/</code>	<code>slaScheduler.js</code> (marks SLA breaches), <code>srAutoApprove.js</code> (auto-approves stuck SR reviews) — run by node-cron
<code>utils/</code>	<code>enrichComplaints.js</code> , <code>jwtHelper.js</code> , <code>responseHelper.js</code> , <code>logger.js</code>

Key endpoints to mention

- `POST /api/auth/google` — the most critical endpoint (unblocks everything).

- `POST /api/complaints` — create complaint (multipart: text fields + `media` photos). Backend auto-resolves `departmentId` from the category if the client omits it.
- `POST /api/complaints/:id/resolve` — staff uploads proof, status → RESOLVED.
- `POST /api/complaints/:id/verify-resolution` — admin approves (→COMPLETED) or sends back (→IN_PROGRESS).
- `GET /api/complaints/export` — **admin-only Excel (.xlsx) export** via ExcelJS, with filters.
- `GET /api/analytics/summary` — dashboard stats.

`enrichComplaints.js` — explain this helper

Raw database rows only have IDs (`categoryId`, `departmentId`). This helper **joins** the relations and fills in the display names (`categoryName` , `departmentName` , `submittedByName` , `assignedToName`) and `photoUrls` / `proofUrls` that the Flutter UI expects. Every complaint-listing endpoint runs through it.

The AI Proxy pattern (`aiProxy.js` + `routes/ai.js`)

The backend's `/api/ai/*` routes forward to the Python service with a **try/catch that returns safe defaults**. This is the integration boundary between Prem's and Pramath's work — and it means an AI outage never breaks the form.

Database (Prisma schema) — main tables

`User` , `Complaint` , `MediaItem` (with `purpose` : ORIGINAL or PROOF), `ComplaintUpdate` (status-change timeline), `Department` , `Category` , `Zone` , `Tag` , `AllowedDomain` , `RefreshToken` . Complaint status is a **free-text string** (not an enum).

Likely exam questions

- *What is Prisma?* → A type-safe ORM that maps JS objects to Postgres tables and handles migrations.
- *Why JWT + refresh token?* → Access token is short-lived (security); refresh token silently renews it without re-login.
- *What is a cron job here?* → Scheduled background tasks (SLA breach detection, auto-approve).

6. MODULE — Flutter Mobile App (`scms_flutter/`) **Pavan + Prabhava**

One-line pitch: A cross-platform (Android) app built with **Clean Architecture** and the **BLoC** state-management pattern, with an **iOS-clean** design (Apple system-blue, grouped cards, large titles).

Tech stack: Flutter · Dart · flutter_bloc (BLoC + Cubit) · go_router · Dio (HTTP) · Hive (offline drafts) · FlutterSecureStorage (tokens) · Firebase (FCM) · Geolocator · image_picker

Clean Architecture layers (draw this)

```
presentation/ → UI (pages) + BLoC/Cubit state management
|
domain/ → entities + usecases (pure business rules)
|
data/ → models + datasources (remote/local) + repositories
|
core/ → theme, constants, network (Dio), errors, utils
```

Why layers? UI doesn't know about HTTP; business logic doesn't know about JSON. Each layer is testable and swappable.

State management: BLoC pattern (explain it)

- **BLoC = Business Logic Component.** UI sends **Events** → BLoC processes → emits **States** → UI rebuilds.
- **Cubit** = simpler BLoC (methods instead of events).
- One BLoC/Cubit per feature: `auth`, `complaint`, `submit_complaint`, `sr_review`, `analytics`, `all_complaints`.

Networking: `DioClient` (`core/network/dio_client.dart`)

- `_AuthInterceptor` — attaches the Bearer token, auto-refreshes on 401.
- `_UnwrapInterceptor` — strips the backend's `{success, data}` envelope so data sources get the raw payload.

6A. Flutter — Pavan's modules (Foundation + Core + Student flow + AI widgets)

Foundation & Core (everything the whole app depends on): - `main.dart` — app entry, Firebase init, dependency injection (RepositoryProviders + BlocProviders). - `app.dart` — **go_router** with **role-based redirects** (a student can't open admin screens). - `core/theme/` — the full iOS-clean design system (colors, text styles, light + dark themes). - `core/network/dio_client.dart` — the HTTP client with interceptors. - `data/models/` — 10 models (User, Complaint, Analytics, GrammarCorrection, DuplicateCheck...) with `fromJson` / `toJson`. - `data/repositories/` — combine remote + local (e.g. `ComplaintRepository` falls back to a **saved offline draft** when there's no network).

Student-facing screens: - `splash` → `onboarding` (3 slides) → `login` (Google Sign-In). - `home_page` — dashboard with greeting hero, stat tiles, quick actions. - `submit_complaint_page` — the flagship screen (see below). - `my_complaints_page` — list with filter chips + pull-to-refresh. - `complaint_detail_page` — full detail + status timeline + rating. - `rating_page` — star rating after COMPLETED.

The Submit-Complaint flow (Pavan's showpiece — explain in detail): 1. Student types the complaint. A **Cubit debounces by 800ms**, then calls AI grammar-check + categorize **while typing**. 2. **Grammar banner** appears showing suggested corrections (Accept / Dismiss). 3. **AI category suggestion** appears (e.g. "Plumbing 100%") with Accept / Change. 4. **Duplicate warning banner** appears if a similar complaint

exists (tap to view matches). 5. Student takes a photo → **CameraService** captures → **WatermarkService** burns **GPS coordinates + datetime** onto the image using a Flutter Canvas **WatermarkPainter**. 6. Submit → multipart POST to backend.

Services Pavan built: **CameraService**, **WatermarkService** (GPS+time stamp), **LocationService** (Geolocator), **GrammarService**, **StorageService** (offline draft folder).

AI widgets Pavan built: **grammar_correction_banner** (diff view), **duplicate_warning_banner**, **category_selector_widget**, **media_capture_widget**, **sla_timer_widget** (live ticking countdown), plus all the shared common widgets (**ScmsButton**, **ComplaintCard**, **StatusBadge**, etc.) that Prabhava reuses.

6B. Flutter — Prabhava's modules (Staff + SR + Admin + Settings + Notifications)

Staff screens: - **staff_dashboard_page** — assigned complaints, live stats (Assigned / In Progress / Done Today). - **staff_complaint_detail_page** — start work + **"Submit Resolution with Proof"** (photos + notes → **/resolve**), plus an activity timeline.

SR (Student Representative) screens: - **sr_dashboard_page** — pending-review queue with severity filter chips. - **sr_review_detail_page** — **Approve** or **Reject with a reason**.

Admin screens: - **admin_dashboard_page** — analytics: KPIs, department/category charts, recent SLA breaches. - Assign/reassign complaints to staff; **verify resolutions** (Approve / Send back); Excel export.

Settings & shared: - **settings_page** — working theme selector (System/Light/Dark) + notification toggle, backed by **AppPreferences**. - **main_shell.dart** — the bottom-nav shell every role lands in (role dashboard + shared All/Stats/Profile tabs).

Notifications (Prabhava's key service): - **notification_service.dart** — **Firebase Cloud Messaging (FCM)** + local notifications + an in-app banner overlay. - Deep-link navigation: tapping a push notification opens the right complaint (uses the global **navigatorKey**). - Respects the Settings toggle — if notifications are off, banners are suppressed.

Analytics widgets: **stats_card**, **complaints_chart**, plus the **AnalyticsCubit** that loads dashboard summaries.

7. Who Did What — Team Work Division

The team is **4 members**, split by **strict file ownership** (you never edit someone else's files — you request a change instead).

Member	Service / Layer	Ownership
 Pavan (Project Lead)	Flutter — Foundation, Core, Data layer, Student flow, AI widgets, Integration	Scaffolded the whole monorepo (Day 0). Built the entire app foundation everyone depends on: theme, networking (Dio), all data models, repositories, common + complaint widgets, and all student screens (submit/list/detail/rating). Built the AI-facing UX: grammar banner, duplicate banner, camera + GPS watermark pipeline. Owns routing (<code>go_router</code>) and app-wide dependency injection. Did the final integration of everyone's work into <code>main</code> .
 Prabhava	Flutter — Staff, SR, Admin, Settings, Notifications	Built all the role dashboards (Staff, SR, Admin) and their detail screens. Owns the SR approve/reject flow, staff proof-of-resolution submission, admin analytics dashboard + charts. Built the entire notification system (FCM + local + in-app banners with deep-linking) and the Settings screen.
 Prem	Node.js Backend (entire service)	Built the complete REST API : Google OAuth + JWT auth, all complaint CRUD, the multi-role workflow endpoints (resolve, verify-resolution), SR routes, analytics, the AI proxy, file uploads, FCM push service, the Prisma schema + migrations, database seeding, the SLA/auto-approve cron jobs , and the Excel export .
 Pramath	Python AI Service (entire service)	Built the FastAPI microservice with all 4 Gemini-powered endpoints: grammar-check (with word diffs), categorize (JSON mode), embed (768-d vectors), and duplicate detection (pgvector cosine similarity). Designed the fail-safe pattern so AI outages never crash the app.

Collaboration model: - Monorepo with per-person Git branches; nobody commits to `main` directly (PRs only). - `CONTEXT.md` is the single source of truth for progress; `TEAM_WORKDIVISION.md` defines ownership. - Two Flutter developers (Pavan + Prabhava) work in **completely separate folders** to avoid conflicts. Shared files (`app.dart` , `main.dart` , `pubspec.yaml`) are edited only by Pavan, who integrates Prabhava's routes/BLoCs.

8. Quick Feature Checklist (rapid-fire Q&A prep)

Feature	Where	Tech
Google-only login (rvce.edu.in)	Backend <code>auth.js</code> + Flutter <code>login_page</code>	OAuth 2.0 + JWT
Grammar auto-correction	AI <code>grammar.py</code>	Gemini 2.0-flash
Auto-categorization + severity	AI <code>categorize.py</code>	Gemini JSON mode
Duplicate detection	AI <code>duplicate.py</code>	pgvector cosine similarity
GPS + time photo watermark	Flutter <code>WatermarkService</code>	Flutter Canvas
Multi-role approval chain	Backend <code>complaints.js</code>	7-status state machine
Proof-of-resolution photos	Staff detail + <code>/resolve</code>	Multer upload
SLA breach tracking	Backend <code>slaScheduler.js</code>	node-cron
Push notifications	Flutter <code>NotificationService</code>	Firebase FCM
Offline drafts	Flutter <code>ComplaintRepository</code>	Hive
Analytics dashboard	Admin dashboard + <code>analytics.js</code>	Charts
Excel export (admin)	Backend <code>/export</code>	ExcelJS
Ratings	Flutter <code>rating_page</code>	1–5 stars

9. Anticipated Examiner Questions (with answers)

Q: Why three separate services instead of one app? A: Separation of concerns, language fit (Python for AI, Node for APIs, Flutter for UI), independent scaling, and fault isolation — the AI service can fail without breaking complaint submission.

Q: Why does Flutter go through Node.js to reach the AI service? A: One security boundary, one auth layer, and Node.js can add safe fallbacks. Exposing the AI service directly would duplicate auth and create a security hole.

Q: What happens if the AI/Gemini service is down? A: The backend's AI proxy catches the error and returns safe defaults (no corrections, no duplicates). The complaint form works normally — AI is an enhancement, not a dependency.

Q: How do you prevent fake complaints / ensure evidence? A: Every photo is watermarked with **GPS + timestamp** at capture time, and staff must upload **proof photos** to resolve — verified by an admin before closure.

Q: How does duplicate detection actually work? A: We convert each complaint into a 768-dimensional embedding vector using Gemini, store it in Postgres with pgvector, and use cosine similarity to find semantically similar complaints — even with different wording.

Q: What is the BLoC pattern and why use it? A: BLoC separates business logic from UI. The UI dispatches events, the BLoC emits states, the UI rebuilds. It makes the app predictable, testable, and keeps widgets thin.

Q: How is security handled? A: Google OAuth restricted to the campus domain, backend-issued JWTs with short-lived access + refresh tokens, role-based access control middleware, and tokens stored in encrypted secure storage on device.

Q: What was the hardest part? A: (Pick one honestly) — the multi-role workflow state machine with the admin-verification rework loop; or the AI integration with fail-safe fallbacks; or the offline-draft + envelope-unwrapping in Flutter.

Good luck with the exam! Read Sections 1, 2, 3, and 7 until you can recite them without looking. Everything else is depth you pull on when the examiner probes.